

24. Ad Click Aggregator

Interviewer: Design the backend for an **ad click aggregator**. Every time a user clicks an ad anywhere on our network, we get a "click" event. We need to show advertisers a near-real-time dashboard — "ad #123 got 4,532 clicks in the last hour" — and, critically, we **bill advertisers based on these same click counts**. Design it.

Candidate: Okay — so this isn't "build a counter," it's "build a counter that a network of flaky mobile clients will fire *at least once*, sometimes *twice*, and that a billing system will trust with real money." That tension — huge noisy firehose vs. an exact number someone gets invoiced on — is the whole problem. Let me clarify first.

1. Clarifying the requirements

Candidate: A few questions before I size anything: 1. Can the *same* physical click generate more than one HTTP call (retry on a flaky network)? 2. What granularity does the dashboard need — per minute, per hour? 3. Is "near-real-time" okay — dashboard lags reality by 10–30 seconds — or sub-second? 4. If the fast dashboard number and the "true" number ever disagree, which one is allowed to be wrong, and for how long?

Interviewer: Yes — beacons retry on failure, so duplicates are expected and common. Per minute is the finest granularity we need. Near-real-time — seconds to low minutes — is fine for the dashboard. But billing must be **exact**, even if it takes longer to finalize.

Candidate: Good, that last answer tells me the shape of the design: I'll need a **fast, approximate path** for the dashboard and a **slower, exact path** for billing — they don't have to be the same computation. Let me write requirements down.

Functional - Ingest a firehose of `ClickEvent(click_id, ad_id, user_id, timestamp)`. - Aggregate into **per-ad, per-minute counts**. - Serve near-real-time dashboards: "clicks for ad X between time A and B." - Feed **billing** off counts that are guaranteed correct, not just fast.

Non-functional - **Accuracy over raw speed** — a stale-but-correct number beats a fast-but-wrong one, because wrong means someone gets over- or under-charged. - **Duplicate-resistant** — retried beacons must not inflate the count (dedup). - **High throughput, bursty** — a viral ad campaign can 10x traffic in seconds. - **Near-real-time**, not hard-real-time — seconds-to-minutes latency is acceptable.

2. Estimation — sizing the firehose

Candidate: Let me size this so the design targets the right numbers.

Assume 10B clicks/day network-wide.
avg = 10B / 86,400s ≈ 115,000 clicks/sec
peak (big shopping event, ~3x) ≈ 350,000 clicks/sec

Event size ≈ 200 bytes (click_id, ad_id, user_id, ts, metadata)
raw volume = 10B * 200B ≈ 2 TB/day
keep raw events 30 days for audit/replay ≈ 60 TB → object storage, NOT a database

Aggregates: 1M distinct ads * 1440 minutes/day = 1.4B rows/day
each row ~50 bytes (ad_id, minute, count) → ~70 GB/day

=> Raw firehose: ~2 TB/day, 350k events/sec peak, full of duplicates.
Aggregate output: ~70 GB/day, tiny, queryable.
The entire design is about safely shrinking the first into the second
WITHOUT losing or double-counting a single billable click.

Candidate: So this is not a storage-capacity problem — 70GB/day of aggregates is nothing. It's a **correctness-under-concurrency-and-duplication** problem, at 350k events/sec.

3. The naive V1 (and why it dies)

Candidate: Let me state the naive version so we're aligned on why it can't work.

```
[ Browser ] --POST /click--> [ App server ] --> UPDATE ad_counts
                                         SET count = count + 1
                                         WHERE ad_id = 123;
```

Why it dies, two separate ways:

1. **Throughput.** 350k writes/sec on one row per hot ad is the classic hot-row problem — `UPDATE ... WHERE ad_id=123` on a viral ad serializes every writer on one row lock.
2. **Correctness.** This counts **HTTP requests**, not **clicks**. A retried beacon is a second request for the same click — we'd increment twice and overbill the advertiser, with no dedup at all. Throughput and correctness fail independently; fixing one doesn't fix the other.

4. The key idea — decouple ingestion from counting, and count *distinct IDs*, not *requests*

Candidate: Two separate ideas need to combine:

1. **Don't count synchronously in the request path.** Absorb the firehose in a durable buffer, then count asynchronously, off to the side, at whatever pace the counting logic can sustain.
2. **Never count a raw event. Count a deduplicated click_id.** Every click gets one client-generated UUID; a retry reuses the *same* click_id. We track "have I already counted this ID," and if yes, we drop it silently.

at-least-once delivery (client retries) + dedup by click_id
=====

effectively-once counting

This is the realistic, buildable version of "exactly once" – we don't pretend duplicates never happen, we make them harmless.

Ingestion API: a dumb, fast beacon endpoint. `POST /v1/clicks {click_id, ad_id, user_id, ts}` → `202 Accepted` the instant it's durably queued — the browser never waits for it to be counted.

5. Streaming ingestion pipeline — Kafka + a stream processor

Interviewer: Walk me through what actually consumes 350k events/sec and turns it into "ad #123: 812 clicks this minute."

Candidate: Two pieces, doing very different jobs.

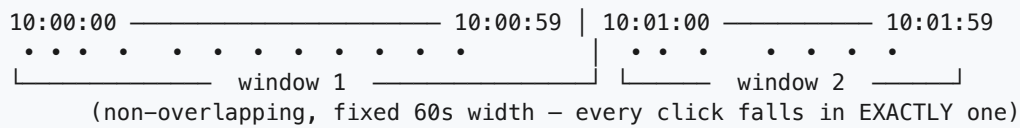
```
graph TD
    A[Browser click] --> B["[ Ingest Service ] (thin, stateless, autoscaled – validates shape, pushes, returns 202)"]
    B --> C["[ Kafka topic: clicks ] partitioned by ad_id, replicated, durable buffer"]
    C --> D["[ Stream Processor cluster ] (Flink / Spark Streaming)"]
    D --> E["[ Aggregate Store ] small, per-ad-per-minute rows – fast for dashboards"]
    D --> F["- dedups by click_id (state per key, see §7)"]
    D --> G["- buckets events into 1-minute TUMBLING windows, grouped by ad_id"]
    D --> H["- emits Aggregate(ad_id, minute, count) when a window closes"]
```

Why Kafka, not a plain queue like SQS, in front: at 350k/s we need (a) **partitioned** parallelism — many consumers reading different slices at once — and (b) a **durable, replayable log**, because we will *need* to replay raw events later for the batch-correction path (§11). SQS gives at-least-once delivery of a work queue, but not cheap bulk replay of history; Kafka's retained, partitioned log is built for exactly this "firehose + later reprocessing" shape.

Why a stream processor (Flink/Spark Streaming), not "a consumer loop with a counter variable": we need **windowing semantics** (tumbling windows, watermarks for late data), **exactly-once state checkpointing** (so a crash doesn't lose in-flight counts or double-apply them), and **built-in keyed-state** for dedup. Rolling this by hand means reinventing checkpointing, exactly-once state, and window management, badly.

Tumbling windows, pictured

ad_id = 123's click stream, plotted on a timeline:



window 1 closes → Aggregate(ad_id=123, minute=10:00, count=13)

window 2 closes → Aggregate(ad_id=123, minute=10:01, count=9)

Tumbling (vs. *sliding*) windows never overlap — one clean number per ad per minute, not a rolling average.

6. Scaling the pipeline — partition by ad_id, and the hot-ad problem

Interviewer: Now push it — a single ad campaign goes viral and pulls in 20% of all traffic. Your ad_id -partitioned pipeline just got one very hot partition.

Candidate: Right — partitioning by `ad_id` is what gives us lock-free scaling in the common case (every worker owns a disjoint set of ad_ids, no cross-worker coordination), but it creates exactly the classic **hot-key** problem for a viral ad.

Normal: ad_id → hash → partition (each ad's clicks all land on ONE partition/worker)
Viral: ad_id=999 alone generates 70k/s → that ONE partition/worker is now the bottleneck

Fix — sub-partition the hot key:

key = ad_id + (random_suffix % K) e.g. K=16 sub-shards for ad_id=999
→ spreads 70k/s across 16 partitions/workers, ~4,400/s each
→ at READ time, sum the K sub-shards back together for ad_id=999's total
(same "split the hot key, merge on read" trick as sharded Redis counters in fixed-inventory problems, applied to streaming aggregation)

We don't sub-shard every ad, only detected hot ones (traffic heuristic or a static allow-list for known big campaigns) — sub-sharding costs an extra merge step on every read.

7. Exactly-once vs. at-least-once — the money conversation

Interviewer: This feeds billing. Walk me through exactly how you guarantee a duplicate click never gets counted twice, end to end — not just "we dedup," the actual mechanism.

Candidate: There are three places a duplicate could sneak back in, and I need a guard at each:

- (1) Client retries the SAME click twice
 - same `click_id` both times → stream processor's dedup state (keyed by `click_id`) has already seen it → second one dropped. Main guard.
- (2) Kafka redelivers a message the stream processor already processed (consumer crashes after processing but before committing offset)
 - same `click_id` arrives again from Kafka's replay → same dedup state catches it. This is WHY dedup keys on `click_id`, not on "have I seen this Kafka offset" – offsets can repeat, `click_ids` are the true identity.
- (3) A stream-processor WORKER crashes and a replacement resumes
 - Flink/Spark checkpoint restores in-flight window state (partial counts + dedup set) from the last checkpoint, so counting resumes from where it left off, not from zero, not double-applied.

The dedup store itself: a rolling window of recently-seen `click_id`s, keyed the same way as the aggregation (per `ad_id` partition, living with the worker that owns that ad). In Flink this is just **keyed operator state**, checkpointed like everything else — no separate external dedup database needed on the hot path. I only need to remember IDs as long as a duplicate could plausibly arrive (a bounded retry window, e.g. a few hours), not forever.

Why not a `UNIQUE(click_id)` constraint on a database write instead? That pushes 350k/s of uniqueness checks onto a database — the same hot-row-at-scale failure as the naive V1. In-memory keyed state does this at memory speed, sharded naturally by the partitioning already in place.

8. Async — the whole design is the async path

Interviewer: What, if anything, is left in the synchronous request path?

Candidate: As little as physically possible: **validate the shape of the JSON, push to Kafka, return** 202. Dedup, windowing, aggregation, the aggregate-store write, and the dashboard cache update all happen asynchronously, arbitrarily far behind the ingest call — the browser firing a beacon must never wait on a stream processor's window logic. Same "ack fast, do the real work off to the side" move as any high-throughput ingestion system, applied to counting.

9. Caching — protecting the aggregate store from dashboard refreshes

Interviewer: Advertisers refresh their dashboard obsessively. How do you keep that from hammering your aggregate store?

Candidate: The access pattern is heavily skewed: a given `(ad_id, minute)` row is written **once** (when its window closes) but potentially **read hundreds of times** as advertisers refresh. Textbook cache case.

Dashboard request: "ad_id=123, last 60 minutes"

[Redis cache] key = (ad_id, time-range, granularity)

hit → return cached JSON

miss → [Aggregate Store] (range-scan minute buckets for ad_id=123)

populate cache (short TTL, e.g. 5–10s) → return to dashboard

Short TTL because the **current** (still-open) minute bucket is still changing; once a window closes, that bucket is immutable and can be cached far more aggressively. This absorbs repeated refreshes without the aggregate store ever seeing 350k/s — only the much smaller volume of *distinct* dashboard queries.

10. Multi-region — count locally, merge globally

Interviewer: Now we're global — clicks happen in the US, EU, and APAC. Design for that.

Candidate: Same principle as any multi-region system: don't force every click to round-trip to one region. Each region ingests and aggregates **its own** clicks first.

Region: US

[Ingest] → [Kafka US] → [Stream Proc US]
→ [Regional Aggregate US]

Region: EU

[Ingest] → [Kafka EU] → [Stream Proc EU]
→ [Regional Aggregate EU]

\
[Global Merge Job]
(sums each region's per-minute counts
into one global total per ad — a
simple SUM, not a second dedup pass)
/

Why the merge doesn't need to re-dedup: each click is generated by exactly one user in exactly one region, ingested by that region's pipeline alone, carrying one `click_id` — never double-ingested across regions. So the global step is pure addition, `sum(region_counts)`, not another correctness problem. This also keeps latency low and helps with data-residency rules.

11. Consistency — exactly-once for billing via lambda architecture

Interviewer: You said the stream path can be "slightly off in rare edge cases." That's not okay for billing. Resolve that.

Candidate: This is the crux of the whole design, so let me be precise about *why* the fast path can drift and what fixes it.

Why the streaming numbers can be slightly wrong, rarely: late-arriving events. A click's network trip can be slow enough that it arrives **after** its minute's window has already closed and been emitted. The stream processor uses a **watermark** — a moving marker meaning "I don't expect events older than this timestamp anymore" — to decide when it's safe to finalize a window. Wait too long and the dashboard feels stale; finalize too early and you occasionally miss a straggler event. There is no watermark setting that's *perfectly* correct and *fast* — that's the fundamental tradeoff.

The fix — lambda architecture: run two paths, use each for what it's good at.

FAST path (speed layer) = the stream processor above.

Near-real-time (seconds), tumbling windows + watermark, GOOD ENOUGH for a dashboard, occasionally imprecise on late-arriving stragglers.

SLOW path (batch layer):

Every raw click is ALSO archived to object storage (\$2, 60TB/30days).

Periodically (hourly/daily), a batch job reads ALL raw events for a time range from object storage and recomputes counts from scratch — no time pressure, no watermark guessing, sees every late event.

This recomputed number is the SOURCE OF TRUTH FOR BILLING.

```

clicks → [ Fast: Flink ] → [ Dashboard: fast, good-enough ]
      ↓
      [ Object Storage, raw, 30d ] → [ Batch recompute, hourly/daily ] → [ Billing: exact ]
  
```

- **Strong / authoritative:** the batch-recomputed number — this is what invoices are cut from.
- **Eventual / approximate:** the streaming number — good enough for "how's my campaign doing right now," never good enough to bill on.
- **Kappa variant** (either answer is fine): instead of a *separate* batch engine, just **replay** the same stream-processing code over the archived raw events when a correction is needed — one engine run twice instead of two engines. Either way: *fast approximate path now, a guaranteed way to recompute correctly later.*

12. Technology choices — what and why

Concern	Choice	Why this, and why not the alternative
Ingest transport	Plain HTTP beacon, 202 Accepted	Caller is any browser/mobile device — needs zero special client support. <i>Not gRPC</i> — no guarantee the client can speak it, and there's no benefit for a fire-and-forget beacon.
Firehose buffer	Kafka, partitioned by ad_id	Durable, replayable log that absorbs 350k/s bursts; retained history enables the batch-recompute path. <i>Not SQS</i> — no cheap bulk replay of history for \$11's batch layer. <i>Not straight to a DB</i> — a relational table can't absorb an unbounded bursty firehose.
Stream processing	Flink / Spark Streaming	Windowing (tumbling), watermarks for late data, keyed state with checkpointing (exactly-once <i>counting</i> semantics), horizontal scaling by partition. <i>Not a hand-rolled consumer loop</i> — you'd reimplement checkpointed state and windowing, badly.

Concern	Choice	Why this, and why not the alternative
Dedup mechanism	Keyed operator state , keyed by <code>click_id</code>	Lives with the partition that owns the ad, checkpointed for free, memory-speed. <i>Not a <code>UNIQUE</code> constraint in a relational DB</i> — pushes 350k/s of uniqueness checks onto a DB row/index, recreating the hot-row bottleneck.
Aggregate store (dashboard reads)	OLAP / wide-column / time-series store (Cassandra, Druid)	The only query is "range-scan minute buckets for this <code>ad_id</code> " — exactly what a time-partitioned, wide-column store serves fast and cheaply at write volume. <i>Not a heavily-indexed relational table</i> — works at small scale, but at 1.4B aggregate rows/day a time-bucketed store is the better fit.
Why not count in a transactional DB directly	—	UPDATE -based counting forces every click through a row lock (hot-row collapse) <i>and</i> gives no natural place to dedup or window. The stream processor gives windowing + dedup + scaling as one coherent mechanism.
Raw event archive	Object storage (S3-class) , ~30 days	Cheap at 2TB/day; exists so the batch/Kappa path has ground truth to recompute from. Not a database — nobody queries this directly, it's replay fuel.
Dashboard cache	Redis, short TTL	Absorbs repeated refreshes; only the still-open current-minute bucket needs a short TTL, closed buckets cache aggressively.
Billing source of truth	Batch recompute (lambda/kappa) over object storage	No watermark guessing, sees every late event, has all the time in the world to be exactly right — the one property billing cannot compromise on.

Say-it-out-loud justification: *"Kafka absorbs the bursty, duplicate-prone click firehose; a stream processor partitioned by `ad_id` does lock-free, near-linear windowed aggregation into per-minute counts, deduplicating by `click_id` so at-least-once delivery becomes effectively-once counting; an OLAP/wide-column store serves the small resulting aggregates fast for dashboards behind a short-TTL cache; and because the streaming path can drift slightly on late data, a lambda-style batch recompute over archived raw events is the actual source of truth billing is cut from — so the system is fast where speed is enough, and exact where money is on the line."*

13. Failure modes & graceful degradation

Failure	What happens	Mitigation
Stream processor worker crashes	In-flight window state and dedup set would be lost	Periodic checkpointing to durable storage; new worker resumes from last checkpoint, re-reads from Kafka's last committed offset — may reprocess a few events, harmless because of <code>click_id</code> dedup
Late / out-of-order events	Click arrives after its minute's window already emitted	Watermark grace period buys some slack; anything still late gets corrected later by the batch layer, never lost (raw event is archived regardless)
Traffic spike / viral ad	One partition/worker overloaded	Kafka buffers the burst (nothing dropped); sub-shard the hot <code>ad_id</code> across K partitions (§6) and merge on read; add processor workers to catch up

Failure	What happens	Mitigation
Kafka broker dies	Risk of losing unread events on that broker	Replication factor 3 — another broker already has the data, no events lost
Aggregate store node dies	Dashboard reads for its shard fail	Replicated (RF=3 style) so reads/writes survive a single node loss
Streaming numbers drift from truth	Rare edge cases (late data, hiccups) leave the fast-path count slightly off	This is expected and tolerated on the dashboard; the batch/lambda recompute is the correction mechanism, and it's what billing reads, never the live stream number

14. Follow-up curveballs

Interviewer: An advertiser wants **hourly** and **daily** totals, not just per-minute.

Candidate: Don't recompute from raw events — roll up the already-small minute buckets. Either compute hour/day sums on read (summing 60 or 1440 tiny rows is cheap), or run a second, cheap aggregation pass that folds minute buckets into hour buckets on a timer. Rides on the same aggregate store, no new pipeline needed.

Interviewer: How do you catch click fraud — bots hammering an ad to drain a competitor's budget?

Candidate: A design in itself, but the hooks are already here: rate-limit per `user_id` /IP at the ingest tier, and feed the same click stream to a separate fraud-scoring pipeline that flags anomalous patterns (implausible click velocity, datacenter IPs). Flagged clicks get excluded from the *billing* aggregate specifically, though still visible in raw logs — I wouldn't let fraud-scoring latency block the dedup/count path itself.

Interviewer: How long can an event be "late" and still count correctly?

Candidate: A direct tuning knob on the watermark's grace period — longer grace means the streaming path waits longer before finalizing a window, trading freshness for fewer stragglers. In practice a few minutes of grace on the fast path, with the batch/Kappa recompute (no grace-period limit — it reads whatever eventually landed in object storage) as the real backstop.

Interviewer: What if the batch recompute *disagrees* with what was already billed?

Candidate: Billing should run on a delay — a day's invoices finalize only after that day's batch recompute, never off streaming numbers. If a correction is still needed later, it becomes an explicit **adjustment line** on the next billing cycle, not a silent overwrite of a past invoice — the same way any billing system handles late corrections, with an audit trail.

15. Deep-dive: "Why not just increment a counter in Postgres per click?"

Interviewer: You've built Kafka plus a stream processor plus windowing — that's a lot of moving parts. Why not the simplest thing: an `ad_counts` table in Postgres, and on every click, `UPDATE ad_counts SET count = count + 1 WHERE ad_id = X`? Just scale Postgres if you need to.

Candidate: It's the right instinct to check, and it's what I ruled out quickly in §3 — but let me go deeper on *why* it fails, because it's not one problem, it's three independent ones stacking on top of each other.

Axis 1 — write volume vs. one row

At 350k clicks/sec peak network-wide, and a single viral ad pulling ~70k/s on its own (§6), every one of those clicks for that ad wants to touch the *same* `ad_counts` row. A row-level `UPDATE` in Postgres serializes behind that row's lock — realistically a few thousand commits/sec ceiling once you account for the WAL fsync and lock-queue wait. 70k/s against a ceiling of a few thousand/sec is the exact same hot-row collapse as the burger giveaway's `promo` row, just with a click firehose instead of a giveaway spike.

Axis 2 — no natural place for dedup

To stop a retried beacon from double-counting, you'd need either a `SELECT 1 FROM seen_clicks WHERE click_id = ?` before the `UPDATE`, or a `UNIQUE(click_id)` constraint that throws a conflict on the retry. Either way, you've turned one hot-row write into **a read plus a conditional write on that same hot row** — the retry that was supposed to be harmless now costs *more* lock contention than the original click, making the bottleneck worse, not better.

Axis 3 — no time bucketing

The dashboard wants "clicks per minute for the last hour." A running counter has no minute boundaries built in — you'd have to hand-roll a `(ad_id, minute)` counter scheme on top, at which point you're re-implementing windowing yourself, minus watermarks, minus late-event handling, minus checkpointed crash recovery. You end up building a worse version of what the stream processor already gives you for free.

Concern	Per-click DB increment (<code>UPDATE ... count+1</code>)	Kafka + Flink windowed aggregation
Throughput ceiling	One row lock caps at roughly 1k–5k commits/sec; nowhere near the 350k/s peak, let alone 70k/s on one viral ad	Kafka partitions + parallel stream workers scale horizontally; sub-shard a hot <code>ad_id</code> (§6) to spread load further
Hot-key behavior	Collapses immediately — every writer for that ad serializes on the same row	Same underlying hot-key risk, but solved by partition sub-sharding and merged on read, not by a single lock
Dedup story	Needs an extra read-or-unique-constraint on the <i>same</i> hot row — doubles contention exactly where you can least afford it	Keyed operator state per <code>click_id</code> , checkpointed, memory-speed, no extra round trip to a shared row
Time bucketing	None built in — must hand-roll a <code>(ad_id, minute)</code> scheme	Tumbling windows + watermarks are native to the processor

Concern	Per-click DB increment (<code>UPDATE ... count+1</code>)	Kafka + Flink windowed aggregation
Failure recovery	Crash mid-transaction just rolls back (safe, but no notion of "resume this in-flight window")	Periodic checkpoints restore in-flight dedup state and partial window counts
Dashboard reads	Would need to scan/aggregate rows on the fly per query	Small, pre-aggregated (<code>ad_id, minute</code>) rows purpose-built for range scans
Operational shape	Looks simpler on a whiteboard	More components, but each one does a job a transactional DB structurally cannot do at this volume

Rule of thumb: A transactional DB increment is fine when writes are low-volume and don't need time-bucketing — a user's profile view counter, say. The moment you have **(a)** a firehose with a single-key hot spot, **(b)** a duplicate-prone at-least-once delivery mechanism, and **(c)** a real need for time-windowed aggregates, you've described precisely the problem stream processing exists to solve. Reach for Kafka plus a windowed stream processor, not a bigger database.

16. Deep-dive: exactly-once billing — a duplicate and a late-arriving event, walked through

Interviewer: Walk me through, concretely, two failure scenarios that could over-charge an advertiser: **(1)** the same click delivered twice, and **(2)** a click that arrives *after* its window has already closed. Show me exactly what happens at each step.

Candidate: Let's take them one at a time — the first is the easy case, the second is where the real risk hides.

Scenario A — the duplicate (at-least-once delivery)

Browser fires a beacon for one physical click, `click_id = c-8841`.
Network hiccups; the browser's retry logic fires it again.

```
t=10:00:12.100  POST /v1/clicks {click_id: c-8841, ad_id: 42, ts: ...}
t=10:00:12.900  POST /v1/clicks {click_id: c-8841, ad_id: 42, ts: ...}
```

└─ SAME click (retry)

Both land in Kafka topic "clicks", partition for `ad_id=42`, as two records.

Stream processor, keyed dedup state for `ad_id=42`:

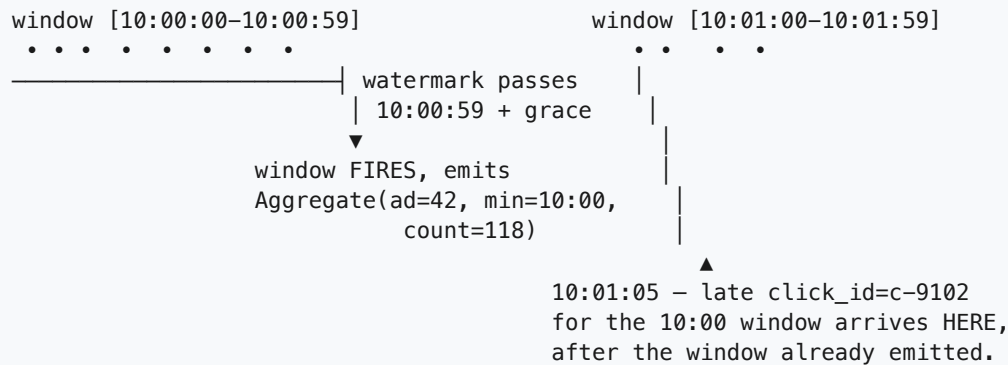
```
record 1: click_id c-8841 -> NOT in dedup state -> count it, remember c-8841
record 2: click_id c-8841 -> ALREADY in dedup state -> drop silently
```

Window `[10:00-10:00:59]` emits: `c-8841` counted exactly ONCE. correct

Dedup-by- `click_id` inside the same window catches this cleanly, as long as both copies arrive before the window's watermark passes and it finalizes.

Scenario B — the late arrival crossing a window boundary

Physical click happens at 10:00:58 (ad_id=42), but the beacon is delayed in flight (slow mobile network) and doesn't reach Kafka until 10:01:05.



Two different bad outcomes, depending on how the pipeline is wired:

- **If late events past a fired window are just dropped:** c-9102 never gets counted in the streaming path — the 10:00 minute is *undercounted* on the dashboard by one. Wrong, but not an overcharge. The overcharge risk shows up in the other branch:
- **If allowed-lateness lets the window re-fire a correction** (the processor re-emits an updated `Aggregate(ad=42, min=10:00, count=119)` once the late event lands), the danger is entirely in *how the downstream sink applies that correction*:

```
1st emission:  count=118  ->  store does  count = count + 118  ->  store = 118
late correction: count=119 (a corrected TOTAL, not a delta!)
                  ->  store does  count = count + 119  ->  store = 237  WRONG
```

The store now shows 237 clicks for a minute that really had 119 — the advertiser gets billed for the sum of two totals, not the true count.

That's the sharp edge case: **the duplicate is easy — dedup catches it. The late-arrival correction interacting with a non-idempotent downstream write is what actually causes silent over-billing**, and it's the kind of bug that only surfaces under real network jitter, well after launch, when nobody's watching that minute closely.

Ranked fixes

1. **Dedup by event-id (click_id), not by Kafka offset or arrival time.** Necessary baseline — fully solves Scenario A. Does *not* solve Scenario B on its own, because the late correction isn't a duplicate event; it's a legitimately new update to an aggregate that already fired.
2. **Watermarks + bounded allowed-lateness.** Hold window state open past the watermark for a bounded grace period (minutes, not hours) so most late events fold into the *original* computation before it ever finalizes, instead of triggering a correction afterward. Reduces how often the risky re-fire path is exercised — never eliminates it, since an event can always be later than the grace period allows.
3. **Idempotent aggregation at the sink.** The actual fix for the bug above: every window emission — first-fire or late correction — must be applied as an **upsert of the final value**, keyed by (ad_id, minute), never as `count = count + delta`. A re-fired correction then simply replaces the row

with the new total; applying it twice has the same effect as applying it once, which is the real meaning of "exactly-once" here.

4. **Lambda/kappa reconciliation as the billing backstop.** Even with 1–3 correct, treat the stream's output as advisory. Every raw click, deduped by `click_id`, is archived to object storage regardless of when it arrived (§11). A batch job — or a Kappa-style replay of the same processing code — recomputes counts for a finalized time range with no watermark guessing at all: it waits until it's certain nothing more is coming, then sees every straggler. **This batch/replay number, not the streaming number, is what invoices are cut from.**

Recommended approach

Do all four, but they're not equally load-bearing. **Click_id dedup and idempotent-upsert aggregation are correctness requirements for the fast path** — get either wrong and even the dashboard is untrustworthy. **Watermark tuning is a latency/completeness knob**, not a correctness guarantee — it only decides how often the late-correction logic gets exercised. And the **lambda/kappa batch recompute over archived raw events is the non-negotiable backstop for billing** — it's the only path with no deadline, so it's the only path allowed to be the source of truth for money changing hands. The fast path should be as correct as reasonably achievable (fixes 1–3); the batch path is the audit that catches everything the fast path couldn't.

What to say out loud: "Deduping by `click_id` solves the easy duplicate. The dangerous case is a late event triggering a corrected aggregate that a non-idempotent sink adds on top of the original instead of replacing — that's the actual over-billing bug, not the duplicate itself. I'd make every aggregate write an idempotent upsert keyed by (`ad_id`, `minute`), use watermarks with bounded allowed-lateness to minimize how often corrections fire at all, and — because no watermark setting is ever perfectly safe — never let billing read the streaming number directly. Billing runs off a batch or Kappa-replay recompute over the archived, deduped raw events, which has no deadline and therefore no late-arrival problem at all."

Summary — the mental model

This problem is **not** "store a lot of events" — 70GB/day of aggregates is trivial. It's "**turn a massive, at-least-once, duplicate-prone firehose into small per-minute numbers that a billing system can trust.**" The winning moves: **Kafka** to durably absorb the bursty ingest; a **stream processor** (Flink/Spark) doing **tumbling-window aggregation partitioned by `ad_id`** for lock-free scaling, with **hot-key sub-sharding** for viral ads; **dedup by client-generated `click_id`** as the single idea that turns at-least-once delivery into effectively-once counting; and — because the fast streaming path can drift on late data — a **lambda/kappa split**: the stream is the fast, good-enough number for dashboards, and a **batch recompute over archived raw events** is the slow, exact number billing actually runs on. Speed and correctness don't have to fight each other if you don't ask one number to be both.